

CHAPTER 2 -PROCESSES MANAGEMENT

The most central concept in any operating system is the *process*: an abstraction of a running program. Everything else hinges on this concept, and it is important that the operating system designer (and student) have a thorough understanding of what a process is as early as possible.

All modern computers can do several things at the same time. While running a user program, a computer can also be reading from a disk and outputting text to a screen or printer. In a multiprogramming system, the CPU also switches from program to program, running each for tens or hundreds of milliseconds. While, strictly speaking, at any instant of time, the CPU is running only one program, in the course of 1 second, it may work on several programs, thus giving the users the illusion of parallelism. We refer to this as **pseudo-parallelism** to contrast it with the true hardware parallelism of **multiprocessor** systems (which have two or more CPUs sharing the same physical memory). Keeping track of multiple, parallel activities is hard for people to do. Therefore, operating system designers over the years have evolved a conceptual model (sequential processes) that makes parallelism easier to deal with. This model, its uses, and some of its consequences form the subject of this discussion.

The Process Model

In this model, all the runnable software on the computer, sometimes including the operating system, is organized into a number of **sequential processes**, or just processes for short. A process is just an executing program, including the current values of the program counter, registers, and variables. Conceptually, each process has its own virtual CPU. In reality, of course, the real CPU switches back and forth from process to process, but to understand the system, it is much easier to think about a collection of processes running in (pseudo) parallel, than to try to keep track of how the CPU switches from program to program. This rapid switching back and forth is called **multiprogramming**.

Figure 2-1 below depicts a computer multiprogramming four programs in memory. There exists only one physical program counter, so when each process runs, its logical program counter is loaded into the real program counter. When it is finished for the time being, the physical program counter is saved in the process' logical program counter in memory.

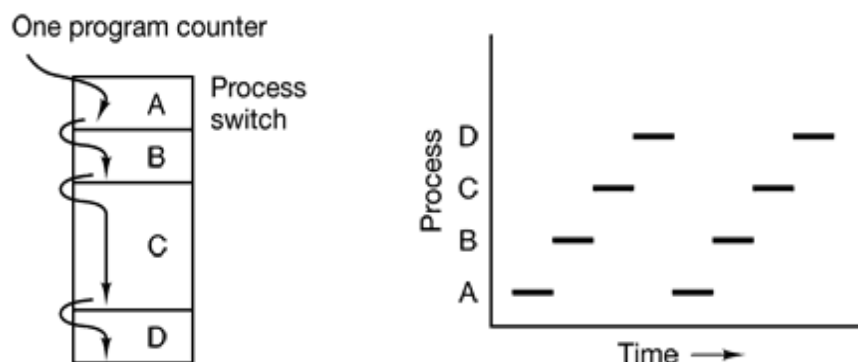


Figure 2-1 Multiprogramming of four programs

With the CPU switching back and forth among the processes, the rate at which a process performs its computation will not be uniform and probably not even reproducible if the same processes are run again. Thus, processes must not be programmed with built-in assumptions about timing.

The difference between a process and a program is subtle, but crucial. An analogy make help here. Consider a culinary-minded computer scientist who is baking a birthday cake for his daughter. He has a birthday cake recipe and a kitchen well stocked with all the input: flour, eggs, sugar, extract of vanilla, and so on. In this analogy, the recipe is the program (i.e., an algorithm expressed in some suitable notation), the computer scientist is the processor (CPU), and the cake ingredients are the input data. The process is the activity consisting of our baker reading the recipe, fetching the ingredients, and baking the cake.

Now imagine that the computer scientist's son comes running in crying, saying that he has been stung by a bee. The computer scientist records where he was in the recipe (the state of the current process is saved), gets out a first aid book, and begins following the directions in it. Here we see the processor being switched from one process (baking) to a higher-priority process (administering medical care), each having a different program (recipe versus first aid book). When the bee sting has been taken care of, the computer scientist goes back to his cake, continuing at the point where he left off.

The key idea here is that a process is an activity of some kind. A process will need certain resources - such as CPU time, memory, files, and I/O devices – to accomplish its task. It is the unit of work in most systems and systems consist of a collection of processes: Operating-system processes execute system code, and user processes execute user code.

The operating system is responsible for the following activities in connection with process management:

- the creation and deletion of both user and system processes
- the scheduling of processes and
- the provision of mechanisms for synchronization, communication, and deadlock handling for processes.

Process Creation

Operating systems need some way to make sure all the necessary processes exist. There are four principal events that cause processes to be created:

1. System initialization.
2. Execution of a process creation system call by a running process.
3. A user request to create a new process.
4. Initiation of a batch job.

When an operating system is booted, typically several processes are created. Some of these are foreground processes, that is, processes that interact with (human) users and perform work for them. Others are background processes, which are not associated with particular users, but

instead have some specific function. For example, one background process may be designed to accept incoming email, sleeping most of the day but suddenly springing to life when email arrives. Another background process may be designed to accept incoming requests for Web pages hosted on that machine, waking up when a request arrives to service the request. Processes that stay in the background to handle some activity such as email, Web pages, news, printing, and so on are called **daemons**. Large systems commonly have dozens of them¹.

In addition to the processes created at boot time, new processes can be created afterward as well. Often a running process will issue system calls to create one or more new processes to help it do its job.

In interactive systems, users can start a program by typing a command or (double) clicking an icon. Taking either of these actions starts a new process and runs the selected program in it.

The last situation in which processes are created applies only to the batch systems found on large mainframes. Here users can submit batch jobs to the system (possibly remotely). When the operating system decides that it has the resources to run another job, it creates a new process and runs the next job from the input queue in it.

Technically, in all these cases, a new process is created by having an existing process execute a process creation system call. That process may be a running user process, a system process invoked from the keyboard or mouse or a batch manager process. What that process does is execute a system call to create the new process. This system call tells the operating system to create a new process and indicates, directly or indirectly, which program to run in it.

In UNIX, there is only one system call to create a new process: `fork`. This call creates an exact clone of the calling process. In Windows, in contrast, a Win32 function call, `CreateProcess`, handles the process creation.

Process Termination

After a process has been created, it starts running and does whatever its job is. However, nothing lasts forever, not even processes. Sooner or later the new process will terminate, usually due to one of the following conditions:

1. Normal exit (voluntary).
2. Error exit (voluntary).
3. Fatal error (involuntary).
4. Killed by another process (involuntary).

Most processes terminate because they have done their work. When a compiler has compiled the program given to it, the compiler executes a system call to tell the operating system that it is

¹ In UNIX, the `ps` program can be used to list the running processes. In Windows systems, the task manager is used.

finished. This call is *exit* in UNIX and *ExitProcess* in Windows. Screen-oriented programs also support voluntary termination. Word processors, Internet browsers and similar programs always have an icon or menu item that the user can click to tell the process to remove any temporary files it has open and then terminate.

The second reason for termination is that the process discovers a fatal error. For example, if a user types the command

```
cc foo.c
```

to compile the program *foo.c* and no such file exists, the compiler simply exits. Screen-oriented interactive processes generally do not exit when given bad parameters. Instead they pop up a dialog box and ask the user to try again.

The third reason for termination is an error caused by the process, often due to a program bug. Examples include executing an illegal instruction, referencing nonexistent memory, or dividing by zero. In some systems (e.g. UNIX), a process can tell the operating system that it wishes to handle certain errors itself, in which case the process is signaled (interrupted) instead of terminated when one of the errors occurs.

The fourth reason a process might terminate is that a process executes a system call telling the operating system to kill some other process. In UNIX this call is *kill*. The corresponding Win32 function is *TerminateProcess*. In both cases, the killer must have the necessary authorization to do in the killing. In some systems, when a process terminates, either voluntarily or otherwise, all processes it created are immediately killed as well. Neither UNIX nor Windows works this way, however.

Process Hierarchies

In some systems, when a process creates another process, the parent process and child process continue to be associated in certain ways. The child process can itself create more processes, forming a process hierarchy. Note that a process has only one parent and zero, one, two, or more children).

In UNIX, a process and all of its children and further descendants together form a process group. When a user sends a signal from the keyboard, the signal is delivered to all members of the process group currently associated with the keyboard (usually all active processes that were created in the current window). Individually, each process can catch the signal, ignore the signal, or take the default action, which is to be killed by the signal.

As another example of where the process hierarchy plays a role, let us look at how UNIX initializes itself when it is started. A special process, called *init*, is present in the boot image. When it starts running, it reads a file telling how many terminals there are. Then it forks off one new process per terminal. These processes wait for someone to log in. If a login is successful, the login process executes a shell to accept commands. These commands may start up more

processes, and so forth. Thus, all the processes in the whole system belong to a single tree, with *init* at the root.

In contrast, Windows does not have any concept of a process hierarchy. All processes are equal. The only place where there is something like a process hierarchy is that when a process is created, the parent is given a special token (called a **handle**) that it can use to control the child. However, it is free to pass this token to some other process, thus invalidating the hierarchy. Processes in UNIX cannot disinherit their children.

Process States

At any point in time, a process can be in any of the following three states:

- *Running* (actually using the CPU at that instant).
- *Ready* (runnable but temporarily stopped to let another process run).
- *Blocked* (unable to run until some external event happens).

Logically, the first two states are similar. In both cases the process is willing to run, only in the second one, there is temporarily no CPU available for it. The third state is different from the first two in that the process cannot run, even if the CPU has nothing else to do.

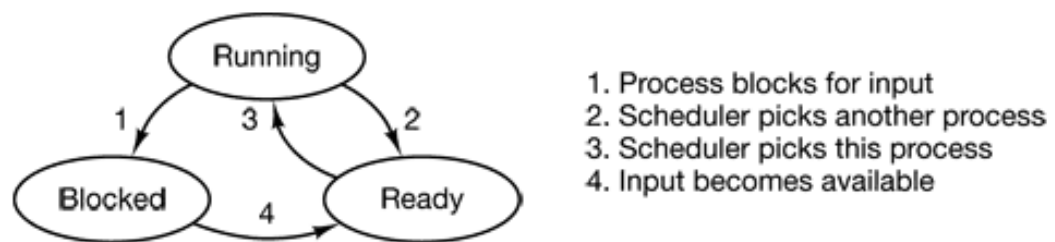


Figure 2-2 A process can be in running, blocked, or ready state

Four transitions are possible among these three states, as shown. Transition 1 occurs when a process discovers that it cannot continue. In some systems the process must execute a system call, such as block or pause, to get into blocked state. In other systems, including UNIX, when a process reads from a pipe or special file (e.g., a terminal) and there is no input available, the process is automatically blocked.

Transitions 2 and 3 are caused by the process scheduler, a part of the operating system, without the process even knowing about them. Transition 2 occurs when the scheduler decides that the running process has run long enough, and it is time to let another process have some CPU time.

Transition 3 occurs when all the other processes have had their fair share and it is time for the first process to get the CPU to run again. The subject of scheduling, that is, deciding which process should run when and for how long, is an important one and many algorithms have been devised to try to balance the competing demands of efficiency for the system as a whole and fairness to individual processes.

Transition 4 occurs when the external event for which a process was waiting (such as the arrival of some input) happens. If no other process is running at that instant, transition 3 will be triggered and the process will start running. Otherwise it may have to wait in *ready* state for a little while until the CPU is available and its turn comes.

Implementation of Processes

To implement the process model, the operating system maintains a table (an array of structures), called the **process table** (*also called process control block*), with one entry per process. This entry contains information about the process':

- state
- program counter
- stack pointer,
- memory allocation
- status of open files
- its accounting and scheduling information, and everything else about the process that must be saved when the process is switched from *running* to *ready* or *blocked* state so that it can be restarted later as if it had never been stopped.

Figure 2-3 shows some of the more important fields in a typical system. The fields in the first column relate to process management. The other two columns relate to memory management and file management, respectively. It should be noted that precisely which fields the process table has is highly system dependent, but this figure gives a general idea of the kinds of information needed.

Process management	Memory management	File management
Registers Program counter Program status word Stack pointer Process state Priority Scheduling parameters Process ID Parent process Process group Signals Time when process started CPU time used Children's CPU time Time of next alarm	Pointer to text segment Pointer to data segment Pointer to stack segment	Root directory Working directory File descriptors User ID Group ID

Figure 2-3 Some of the fields of a typical process table entry

Context Switch

Switching the CPU to another process requires saving the state of the old process and loading the saved state for the new process. This task is known as a ***context switch***. Context-switch time is pure overhead, because the system does no useful work while switching. Its speed varies from machine to machine, depending on the memory speed, the number of registers which must be copied, and the existence of special instructions (such as a single instruction to load or store all registers). Typically, the speed ranges from 1 to 1000 microseconds.

The Thread Model

One way of looking at a process is that it is way to group related resources together. A process has an address space containing program text and data, as well as other resources. These resources may include open files, child processes, pending alarms, signal handlers, accounting information, and more. By putting them together in the form of a process, they can be managed more easily.

The other concept a process has is a thread of execution, usually shortened to just **thread**. The thread has a program counter that keeps track of which instruction to execute next. It has registers, which hold its current working variables. It has a stack, which contains the execution history, with one frame for each procedure called but not yet returned from. Although a thread must execute in some process, the thread and its process are different concepts and can be treated separately. Processes are used to group resources together; threads are the entities scheduled for execution on the CPU.

What threads add to the process model is to allow multiple executions to take place in the same process environment, to a large degree independent of one another. Having multiple threads

running in parallel in one process is analogous to having multiple processes running in parallel in one computer. In the former case, the threads share an address space, open files, and other resources. In the latter case, processes share physical memory, disks, printers, and other resources. Because threads have some of the properties of processes, they are sometimes called **lightweight processes**. The term **multithreading** is also used to describe the situation of allowing multiple threads in the same process.

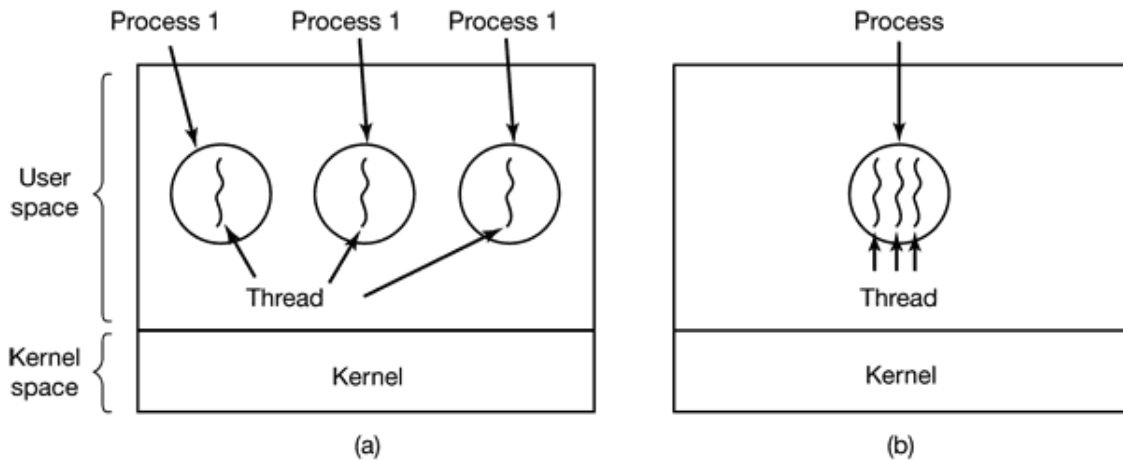


Figure 2- 4 (a) three processes each with one thread. (b) One process with three threads.

In Fig. 2-4(a) we see three traditional processes. Each process has its own address space and a single thread of control. In contrast, in Fig. 2-4(b) we see a single process with three threads of control. Although in both cases we have three threads, in Fig. 2-4(a) each of them operates in a different address space, whereas in Fig. 2-4(b) all three of them share the same address space.

When a multithreaded process is run on a single-CPU system, the threads take turns running. The CPU switches rapidly back and forth among the threads providing the illusion that the threads are running in parallel, albeit on a slower CPU than the real one. With three compute-bound threads in a process, the threads would appear to be running in parallel, each one on a CPU with one-third the speed of the real CPU.

Different threads in a process are not quite as independent as different processes. All threads have exactly the same address space, which means that they also share the same global variables. Since every thread can access every memory address within the process' address space, one thread can read, write, or even completely wipe out another thread's stack. There is no protection between threads because it is impossible, and it should not be necessary. Unlike different processes, which may be from different users and which may be hostile to one another, a process is always owned by a single user, who has presumably created multiple threads so that they can cooperate, not fight. In addition to sharing an address space, all the threads share the same set of open files, child processes, alarms, and signals, etc. as shown in Table 2.1. Thus the organization of Fig. 2-4(a) would be used when the three processes are essentially unrelated, whereas Fig. 2-4(b) would be appropriate when the three threads are actually part of the same job and are actively and closely cooperating with each other.

Table 2.1 Items per process and per thread

Per process items	Per thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

The items in the first column are process properties, not thread properties. For example, if one thread opens a file, that file is visible to the other threads in the process and they can read and write it. This is logical since the process is the unit of resource management, not the thread. If each thread had its own address space, open files, pending alarms, and so on, it would be a separate process. *What we are trying to achieve with the thread concept is the ability for multiple threads of execution to share a set of resources so they can work together closely to perform some task.*

Like a traditional process (i.e., a process with only one thread), a thread can be in any one of several states: running, blocked, ready, or terminated. A running thread currently has the CPU and is active. A blocked thread is waiting for some event to unblock it. For example, when a thread performs a system call to read from the keyboard, it is blocked until input is typed. A thread can block waiting for some external event to happen or for some other thread to unblock it. A ready thread is scheduled to run and will as soon as its turn comes up. Also, thread creation and termination is very much like process creation and termination, with approximately the same options as well.

When multithreading is present, processes normally start with a single thread present. This thread has the ability to create new threads by calling a library procedure. Sometimes threads are hierarchical, with a parent-child relationship, but often no such relationship exists, with all threads being equal.

Why Threads?

We need to use threads for many reasons. The major advantages attained by using threads are:

- In many applications, multiple activities are going on at once. Some of these may block from time to time. By decomposing such an application into multiple sequential threads that run in quasi-parallel, the programming model becomes simpler.
- Since threads do not have any resources attached to them, they are easier to create and destroy than processes. In many systems, creating a thread goes 100 times faster than creating a process. When the number of threads needed changes dynamically and rapidly, this property is useful.

It is probably easiest to see why threads are useful by giving some concrete examples. As a first example, consider a word processor. Most word processors display the document being created on the screen formatted exactly as it will appear on the printed page. In particular, all the line breaks and page breaks are in their correct and final position so the user can inspect them and change the document if need be .

Now consider what happens when the user suddenly deletes one sentence from page 1 of an 800-page document. After checking the changed page to make sure it is correct, the user now wants to make another change on page 600 and types in a command telling the word processor to go to that page. The word processor is now forced to reformat the entire book up to page 600 on the spot and this may bear a substantial delay before page 600 can be displayed, leading to an unhappy user.

Threads can help here. Suppose that the word processor is written as a three -threaded program. One thread interacts with the user and the other handles reformatting in the background. As soon as the sentence is deleted from page 1 the interactive thread tells the reformatting thread to reformat the whole book. Meanwhile, the interactive thread continues to listen to the keyboard and mouse and responds to simple commands like scrolling page 1 while the other thread is computing madly in the background. With a little luck, the reformatting will be completed before the user asks to see page 600, so it can be displayed instantly. The third thread can be used to handle automatically saving the entire file to disk every few minutes to protect the user against losing a day's work in the event of a program crash, system crash, or power failure.

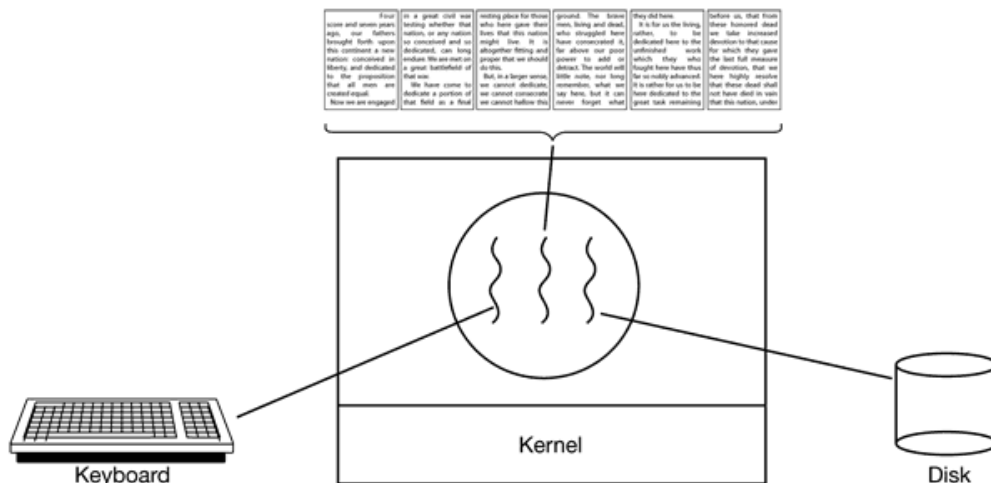


Figure 2-6 A word processor with three threads.

It should be clear that having three separate processes would not work here because all three threads need to operate on the document. By having three threads instead of three processes, they share a common memory and thus all have access to the document being edited.

An analogous situation exists with many other interactive programs. For example, an electronic spreadsheet is a program that allows a user to maintain a matrix, some of whose elements are

data provided by the user. Other elements are computed based on the input data using potentially complex formulas. When a user changes one element, many other elements may have to be recomputed. By having a background thread do the *recompute*, the interactive thread can allow the user to make additional changes while the computation is going on. Similarly, a third thread can handle periodic backups to disk on its own.

Now consider yet another example of where threads are useful: a server for a World Wide Web site. Requests for pages come in and the requested page is sent back to the client. One way to organize the Web server is shown in Fig. 2-7(a). Here one thread, the **dispatcher**, reads incoming requests for work from the network. After examining the request, it chooses an idle (i.e., blocked) **worker thread** and hands it the request. The dispatcher then wakes up the sleeping worker, moving it from blocked state to ready state.

When the worker wakes up, it checks to see if the request can be satisfied from the Web page cache², to which all threads have access. If not, it starts a read operation to get the page from the disk and blocks until the disk operation completes. When the thread blocks on the disk operation, another thread is chosen to run, possibly the dispatcher, in order to acquire more work, or possibly another worker that is now ready to run.

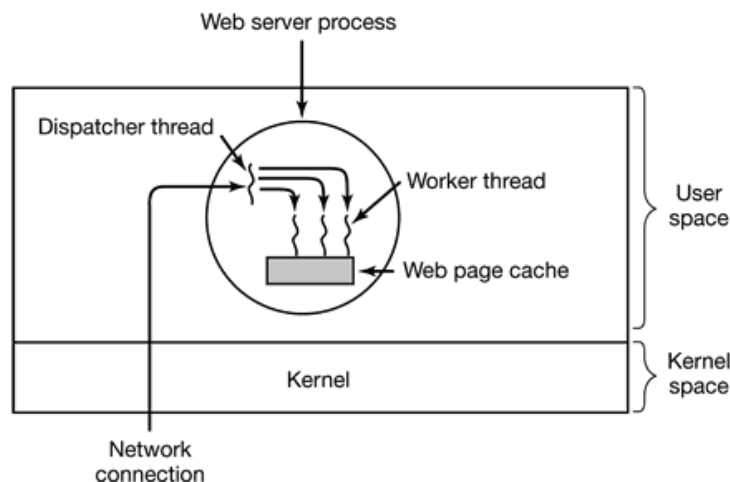


Figure 2-7 A multi threaded Web server

INTERPROCESS COMMUNICATION (IPC)

Processes frequently need to communicate with other processes. For example, in a shell pipeline like the one shown below, the output of the first process must be passed to the second process.

```
cat chapter1 chapter2 chapter3 | grep tree
```

Here, the first process, running *cat*, concatenates and outputs three files. The second process, running *grep*, selects all lines containing the word “tree.” In this example *grep* should wait until *cat* finishes. Other types of processes might have other requirements.

² Web page cache contains Web pages that are more commonly accessed than other pages

IPC provides a mechanism to allow processes to communicate and to synchronize their actions. Very briefly, IPC deals with three issues. These are:

1. *Facilitating passing of information among processes*
2. *Making sure two or more processes do not get into each other's way when engaging in critical activities (suppose two processes each try to grab the last 1 MB of memory).*
3. *Sequencing processes properly when dependencies are present (if process A produces data and process B prints them, B has to wait until A has produced some data before starting to print).*

The first one—passing information—is easy for threads since they share a common address space. However, the other two—keeping out of each other's hair and proper sequencing—apply equally well to threads. The same problems exist and the same solutions apply. Below we will discuss the problem in the context of processes, but please keep in mind that the same problems and solutions also apply to threads.

Race Conditions , Critical sections, & Mutual Exclusion

In some operating systems, processes that are working together may share some common storage that each one can read and write. The shared storage may be in main memory or it may be a shared file. This may bring about a problem as described below.

Let us consider a simple but common example: a print spooler. When a process wants to print a file, it enters the file name in a special **spooler directory**. Another process, the **printer daemon**, periodically checks to see if there are any files to be printed, and if there are, it prints them and then removes their names from the directory.

Imagine that our spooler directory has a very large number of slots, numbered $0, 1, 2, \dots, n$ each one capable of holding a file name. Also imagine that there are two shared variables, **out**, which points to the next file to be printed, and **in**, which points to the next free slot in the directory. At a certain instant, slots 0 to 3 are empty (the files have already been printed) and slots 4 to 6 are full (with the names of files queued for printing). More or less simultaneously, processes A and B decide they want to queue a file for printing. This situation is shown in Fig. 2-8.

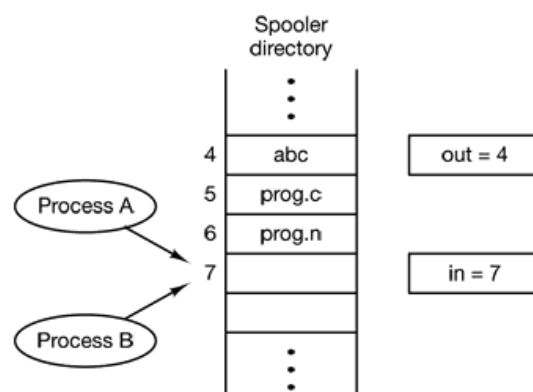


Figure 2-8. Two processes trying to access shared memory at the same time

The following might happen.

- Process *A* reads *in* and stores the value, 7, in a local variable called *next_free_slot*. Just then a clock interrupt occurs and the CPU decides that process *A* has run long enough, so it switches to process *B*,
- Process *B* also reads *in*, and also gets a 7. It too stores it in *its* local variable *next_free_slot*. At this instant both processes think that the next available slot is 7.
- Process *B* now continues to run. It stores the name of its file in slot 7 and updates *in* to be 8. Then it goes off and does other things.
- Eventually, process *A* runs again, starting from the place it left off. It looks at *next_free_slot*, finds a 7 there, and writes its file name in slot 7, erasing the name that process *B* just put there. Then it computes *next_free_slot* + 1, which is 8, and sets *in* to 8.

The spooler directory is now internally consistent, so the printer daemon will not notice anything wrong, but process *B* will never receive any output. User *B* will hang around the printer room for years, wistfully hoping for output that never comes. ***Situations like this, where two or more processes are reading or writing some shared data and the final result depends on who runs precisely when, are called race conditions.***

Part of a program that accesses the shared memory is called the ***critical region*** or ***critical section***.

If we could arrange matters such that no two processes were ever in their critical regions at the same time, we could avoid races.

Making sure that if one process is using a shared variable or file, the other processes will be excluded from doing the same thing is referred to as ***mutual exclusion***.

In an abstract sense, mutual exclusion is shown in Fig. 2-9. Here process *A* enters its critical region at time T_1 , A little later, at time T_2 process *B* attempts to enter its critical region but fails because another process is already in its critical region and we allow only one at a time. Consequently, *B* is temporarily suspended until time T_3 when *A* leaves its critical region, allowing *B* to enter immediately. Eventually *B* leaves (at T_4) and we are back to the original situation with no processes in their critical regions.

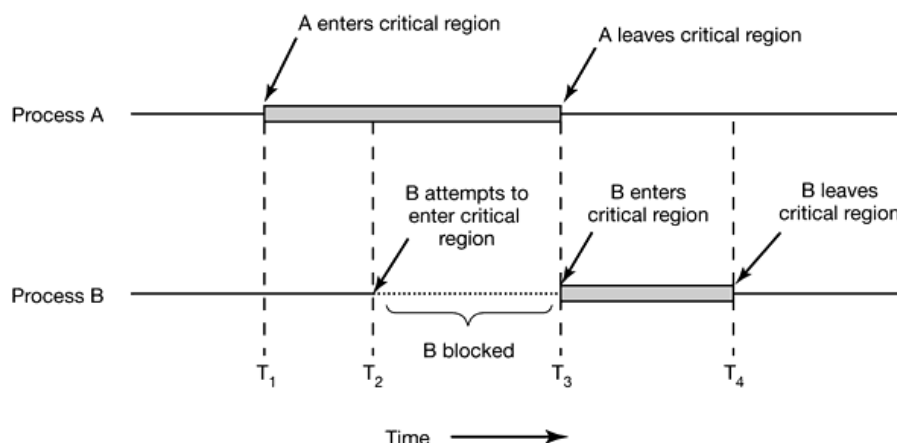


Figure 2-9 Mutual exclusion using critical regions

Although mutual exclusion avoids race conditions, it is not sufficient for having parallel processes cooperate correctly and efficiently using shared data. We need four conditions to hold to have a good solution:

- A. No two processes may be simultaneously inside their critical regions
- B. No assumptions may be made about speeds or the number of CPUs
- C. No process running outside its critical region may block other processes
- D. No process should have to wait forever to enter its critical region

Techniques for achieving mutual exclusion

Mutual Exclusion with Busy Waiting

1. **Disabling Interrupts** - have each process disable all interrupts just after entering its critical region and re-enable them just before leaving it. With interrupts turned off the CPU will not be switched to another process. Thus, the process can examine and update the shared memory without fear that any other process will intervene.

Limitations:

- It is unwise to give user processes the power to turn off interrupts. If one of the processes did it and never turned them on again, that could be the end of the system.
- If the system is a multiprocessor, with two or more CPUs, disabling interrupts affects only the CPU that executed the disable instruction. The other ones will continue running and can access the shared memory.

2. **Lock Variables** – have a single, shared (lock) variable, initially 0. When a process wants to enter its critical region, it first tests the lock. If the lock is 0, the process sets it to 1 and enters the critical region. If the lock is already 1, the process just waits until it becomes 0. Thus, a 0 means that no process is in its critical region, and a 1 means that some process is in its critical region.

Limitations

- same fatal flaw that we saw in the spooler directory. Suppose that one process reads the lock and sees that it is 0. Before it can set the lock to 1, another process is scheduled, runs, and sets the lock to 1. When the first process runs again, it will also set the lock to 1, and two processes will be in their critical regions at the same time.
3. **Strict Alternation** – use an integer variable *turn*, initially 0, to keep track of whose turn it is to enter the critical region and examine or update the shared memory. Initially, process 0 inspects *turn*, finds it to be 0, and enters its critical region. Process 1 also finds it to be 0 and therefore sits in a tight loop continually testing *turn* to see when it becomes 1. When process 0 leaves the critical region, it sets *turn* to 1, to allow process 1 to enter its critical region.

```
while (TRUE) {           while (TRUE) {
    while (turn != 0) /* loop while (turn != 1); /* loop */ ;
    /* ;                  critical_region();
    critical_region();    turn = 0;
    turn = 1;             noncritical_region();
    noncritical_region(); }
}
```

Process 0

Process 1

Limitations

- Taking turns is not a good idea when one of the processes is much slower than the other. Suppose that process 1 finishes its critical region quickly, so both processes are in their noncritical regions, with *turn* set to 0. Now process 0 executes its whole loop quickly, exiting its critical region and setting *turn* to 1. At this point *turn* is 1 and both processes are executing in their noncritical regions. Suddenly, process 0 finishes its noncritical region and goes back to the top of its loop. Unfortunately, it is not permitted to enter its critical region now, because *turn* is 1 and process 1 is busy with its noncritical region. It hangs in its while loop until process 1 sets *turn* to 0. This situation violates condition C set out above: process 0 is being blocked by a process not in its critical region.

4. **Peterson's Algorithm** - consists of two procedures written in ANSI C that uses a variable *turn* to indicate whose turn it is to use the shared data. Before using the shared variables (i.e., before entering its critical region), each process calls *enter_region* with its own process number, 0 or 1, as parameter. This call will cause it to wait, if need be, until it is safe to enter. After it has finished with the shared variables, the process calls *leave_region* to indicate that it is done and to allow the other process to enter, if it so desires.

```
#define FALSE 0
#define TRUE 1
#define N 2 /* number of processes */

int turn; /* whose turn is it? */
int interested[N]; /* all values initially 0 (FALSE) */

void enter_region(int process) /* process is 0 or 1 */
{
    int other; /* number of the other process */

    other = 1 - process; /* the opposite of process */
    interested[process] = TRUE; /* show that you are interested */
    turn = process; /* set flag */
    while (turn == process && interested[other] == TRUE) /* null statement */;
}

void leave_region (int process) /* process, who is leaving */
{
    interested[process] = FALSE; /* indicate departure from critical region */
}
```

5. **The TSL Instruction** – is a proposal that requires a little help from the hardware. Many computers, especially those designed with multiple processors in mind, have an instruction TSL RX , LOCK (Test and Set Lock) that works as follows. It reads the contents of the memory word

LOCK into register RX and then stores a nonzero value at the memory address lock where the operations of reading the word and storing into it are guaranteed to be indivisible—no other processor can access the memory word until the instruction is finished.

When lock is 0, any process may set it to 1 using the TSL instruction and then read or write the shared memory. When it is done, the process sets lock back to 0 using an ordinary move instruction. This technique can be implemented as a four-instruction subroutine in a fictitious (but typical) assembly language shown below.

```
enter_region:
    TSL REGISTER,LOCK    // copy lock to register and set lock to 1
    CMP REGISTER,#0      // was lock zero?
    JNE enter_region     // if it was non zero, lock was set, so loop
    RET                  // return to caller; critical region entered

leave_region:
    MOVE LOCK,#0         // store a 0 in lock
    RET                  // return to caller
```

Mutual Exclusion without Busy Waiting

All the above techniques have the defect of requiring busy waiting: when a process wants to enter its critical region, it checks to see if the entry is allowed. If it is not, the process just sits in a tight loop waiting until it is.

This approach results in two problems. It

- Wastes CPU time
- Causes priority inversion problem. Consider a computer with two processes, *H*, with high priority and *L*, with low priority. The scheduling rules are such that *H* is run whenever it is in ready state. At a certain moment, with *L* in its critical region, *H* becomes ready to run (e.g., an I/O operation completes). *H* now begins busy waiting, but since *L* is never scheduled while *H* is running, *L* never gets the chance to leave its critical region, so *H* loops forever. This situation is sometimes referred to as the **priority inversion problem**.

It is therefore important to come up with a better solution, let us look at some interprocess communication primitives that block instead of wasting CPU time when they are not allowed to enter their critical regions. Below, we will discuss some of those types.

However, before going directly to the discussion of the techniques, let's set a scenario, referred to as *The Producer-Consumer Problem* to show how these primitives can be used.

The Producer-Consumer Problem

As an example of, let us consider the producer-consumer problem (also known as the bounded-**buffer** problem). Two processes share a common, fixed-size buffer. One of them, the producer, puts information into the buffer, and the other one, the consumer, takes it out.

Trouble arises when the producer wants to put a new item in the buffer, but it is already full. The solution is for the producer to be blocked to be resumed when the consumer has removed one or

more items. Similarly, if the consumer wants to remove an item from the buffer and sees that the buffer is empty, it blocks until the producer puts something in the buffer and wakes it up. This approach sounds simple enough, but it leads to the same kinds of race conditions we saw earlier with the spooler directory.

1. **Sleep and Wakeup** – uses two system calls called `sleep` and `wakeup`. `Sleep` causes the caller to block, that is, be suspended until another process wakes it up. The `wakeup` call has one awakens a process blocked by `sleep`.

To keep track of the number of items in the buffer, we will need a variable, *count*. If the maximum number of items the buffer can hold is *N*, the producer's code will first test to see if *count* is *N*. If it is, the producer will go to sleep; if it is not, the producer will add an item and increment *count*.

The consumer's code is similar: first test *count* to see if it is 0. If it is, go to sleep, if it is nonzero, remove an item and decrement the counter. Each of the processes also tests to see if the other should be awakened, and if so, wakes it up. The code for both producer and consumer is shown in

```
#define N 100      /* number of slots in the buffer */
int count = 0;    /* number of items in the buffer */

void producer(void)
{
    int item;

    while (TRUE) {          /* repeat forever */
        item = produce_item(); /* generate next item */
        if (count == N) sleep(); /* if buffer is full, go to sleep */
        insert_item(item);    /* put item in buffer */
        count = count + 1;    /* increment count of items in buffer */
        if (count == 1) wakeup(consumer); /* was buffer empty? */
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {          /* repeat forever */
        if (count == 0) sleep(); /* if buffer is empty, got to sleep */
        item = remove_item(); /* take item out of buffer */
        count = count - 1;    /* decrement count of items in buffer */
        if (count == N - 1) wakeup(producer); /* was buffer full? */
        consume_item(item);   /* print item */
    }
}
```

Limitation:

- Race condition can occur because access to *count* is unconstrained. The following situation could possibly occur. The buffer is empty and the consumer has just read *count* to see if it is 0. At that

instant, the scheduler decides to stop running the consumer temporarily and start running the producer. The producer inserts an item in the buffer, increments *count*, and notices that it is now 1. Reasoning that *count* was just 0, and thus the consumer must be sleeping, the producer calls *wakeup* to wake the consumer up.

Unfortunately, the consumer is not yet logically asleep, so the wakeup signal is lost. When the consumer next runs, it will test the value of *count* it previously read, find it to be 0, and go to sleep. Sooner or later the producer will fill up the buffer and also go to sleep. Both will sleep forever.

2. **Semaphores** – uses an integer variable (semaphore) to count the number of wakeups saved for future use. A semaphore could have the value 0, indicating that no wakeups were saved, or some positive value if one or more wakeups were pending.

Two operations, down and up (generalizations of sleep and wakeup, respectively) would be performed on this variable. The down operation on a semaphore checks to see if the value is greater than 0.

- *If so, it decrements the value (i.e., uses up one stored wakeup) and just continues.*
- *If the value is 0, the process is put to sleep without completing the down for the moment. Checking the value, changing it and possibly going to sleep, is all done as a single, indivisible **atomic action**. This atomicity is absolutely essential to solving synchronization problems and avoiding race conditions.*

The up operation increments the value of the semaphore addressed. If one or more processes were sleeping on that semaphore, unable to complete an earlier down operation, one of them is chosen by the system (e.g., at random) and is allowed to complete its down.

Solving the Producer-Consumer Problem using Semaphores

```
#define N 100          /* number of slots in the buffer */

typedef int semaphore; /* semaphores are a special kind of int */
semaphore mutex = 1;   /* controls access to critical region */
semaphore empty = N;   /* counts empty buffer slots */
semaphore full = 0;    /* counts full buffer slots */

void producer(void)
{
    int item;

    while (TRUE) {      /* TRUE is the constant 1 */
        item = produce_item(); /* generate something to put in buffer */
        down(&empty);      /* decrement empty count */
        down(&mutex);      /* enter critical region */
        insert_item(item); /* put new item in buffer */
        up(&mutex);        /* leave critical region */
        up(&full);         /* increment count of full slots */
    }
}

void consumer(void)
{
    int item;
```

```

while (TRUE) {          /* infinite loop */
    down(&full);          /* decrement full count */
    down(&mutex);         /* enter critical region */
    item a= remove_item(); /* take item from buffer */
    up(&mutex);           /* leave critical region */
    up(&empty);           /* increment count of empty slots */
    consume_item(item);   /* do something with the item */
}
}

```

- 3. Monitors -
- 4. Message Passing -
- 5. Barriers -

CLASSICAL IPC PROBLEMS

The operating systems literature is full of interesting problems that have been widely discussed and analyzed using a variety of synchronization methods. Everyone inventing a new IPC primitive usually demonstrates how wonderful the new primitive is by showing how elegantly it solves these common problems. In the following sections we will examine three of the better-known problems.

The Dining Philosophers Problem

Five philosophers are seated around a circular table. Each philosopher has a plate of spaghetti. The spaghetti is so slippery that a philosopher needs two forks to eat it. Between each pair of plates is one fork. The layout of the table is illustrated in Fig. 2-10.

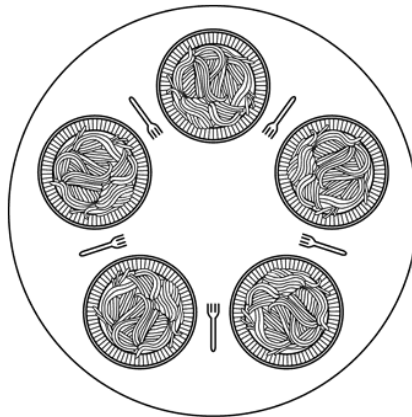


Figure 2-10 Lunch time in the Philosophy Department

The life of a philosopher consists of alternate periods of eating and thinking. When a philosopher gets hungry, she tries to acquire her left and right fork, one at a time, in either order. If successful in acquiring two forks, she eats for a while, then puts down the forks, and continues to think. The key question is: Can you write a program for each philosopher that does what it is supposed to do and never gets stuck? *The dining philosophers problem is useful for modeling processes that are competing for exclusive access to a limited number of resources, such as I/O devices*

The Readers and Writers Problem

The readers and writers problem which models access to a database. Imagine, for example, an airline reservation system, with many competing processes wishing to read and write it. It is acceptable to have multiple processes reading the database at the same time, but if one process is updating (writing) the database, no other processes may have access to the database, not even readers. *The question is how do you program the readers and the writers?*

The Sleeping Barber Problem

Another classical IPC problem takes place in a barber shop. The barber shop has one barber, one barber chair, and n chairs for waiting customers, if any, to sit on. If there are no customers present, the barber sits down in the barber chair and falls asleep. When a customer arrives, he has to wake up the sleeping barber. If additional customers arrive while the barber is cutting a customer's hair, they either sit down (if there are empty chairs) or leave the shop (if all chairs are full). The problem is to program the barber and the customers without getting into race conditions. This problem is similar to various queueing situations, such as a multiperson helpdesk with a computerized call waiting system for holding a limited number of incoming calls.

SCHEDULING

When a computer is multiprogrammed, it frequently has multiple processes competing for the CPU at the same time. If only one CPU is available, a choice has to be made which process to run next. The part of the operating system that makes the choice is called the **scheduler** and the algorithm it uses is called the **scheduling algorithm**.

Because CPU time is a scarce resource on these machines, a good scheduler can make a big difference in perceived performance and user satisfaction. Consequently, a great deal of work has gone into devising clever and efficient scheduling algorithms.

Process Behavior

Nearly all processes alternate bursts of computing with (disk) I/O requests. Processes that spend most of their time computing are called **compute-bound**, while those that spend most of their time waiting for I/O are called **I/O-bound**.

When to Schedule

- When a new process is created
- When a process exits.
- When a process blocks on I/O, on a semaphore, or for some other reason
- When an I/O interrupt occurs

Scheduling algorithms can be divided into two categories with respect to how they deal with clock interrupts – preemptive & non-preemptive

- A **nonpreemptive** scheduling algorithm picks a process to run and then just lets it run until it blocks (either on I/O or waiting for another process) or until it voluntarily releases the CPU. Even if it runs for hours, it will not be forcibly suspended.
- A **preemptive** scheduling algorithm picks a process and lets it run for a maximum of some fixed time. If it is still running at the end of the time interval, it is suspended and the scheduler picks another process to run (if one is available). Doing preemptive scheduling requires having a clock interrupt occur at the end of the time interval to give control of the CPU back to the scheduler. If no clock is available, nonpreemptive scheduling is the only option.

Different environments need different scheduling algorithms. This situation arises because different application areas (and different kinds of operating systems) have different goals (what the scheduler should optimize for is not the same in all systems). For example:

- *In batch systems, there are no users impatiently waiting at their terminals for a quick response. Consequently, nonpreemptive algorithms, or preemptive algorithms with long time periods for each process are often acceptable. This approach reduces process switches and thus improves performance.*
- *In an environment with interactive users, preemption is essential to keep one process from hogging the CPU and denying service to the others.*

Scheduling Algorithm Goals

In order to design a scheduling algorithm, it is necessary to have some idea of what a good algorithm should do. Some goals depend on the environment (batch, interactive, or real time), but there are also some that are desirable in all cases. Some of these goals are:

All systems

- Fairness - giving each process a fair share of the CPU
- Policy enforcement - seeing that stated policy is carried out
- Balance - keeping all parts of the system busy

Batch systems

- Throughput - maximize jobs per hour
- Turnaround time - minimize time between submission and termination
- CPU utilization - keep the CPU busy all the time

Interactive systems

- Response time - respond to requests quickly
- Proportionality - meet users' expectations

Real-time systems

- Meeting deadlines - avoid losing data
- Predictability - avoid quality degradation in multimedia systems

Scheduling in Batch Systems

- **First-Come First-Served** - in this nonpreemptive algorithm, processes are assigned the CPU in the order they request it. Basically, there is a single queue of ready processes. When the first job enters the system, it is started immediately and allowed to run as long as it wants to. As other jobs come in, they are put onto the end of the queue. When the running process blocks, the first process on the queue is run next. When a blocked process becomes ready, like a newly arrived job, it is put on the end of the queue.

Strength

- Easy to understand and equally easy to program.

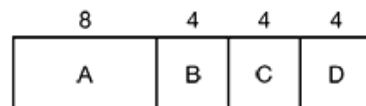
Limitations

- Performs much better for long processes than short ones. For example, in the below shown list of processes, P3 is in the system for a very long period of time (100 times its required processing time)

Process	Arrival time	Service time (sec)	Start time	Finish time	Turnaround time
P1	0	1	0	1	1
P2	1	100	1	101	100
P3	2	1	101	102	100
P4	3	100	102	202	199

- Tends to favor processor-bound processes over I/O bound processes

- **Shortest Job First** – is a nonpreemptive batch algorithm , that assumes the run times are known in advance, that picks the shortest job first. When all the jobs are available simultaneously, shortest job first is optimal. Consider the following four jobs A,B,C, and D with the indicated run times.



(a)



(b)

<i>Turnaround time using FCFS (a)</i>	
<i>A</i>	8 minutes
<i>B</i>	12 minutes
<i>C</i>	16 minutes
<i>D</i>	20 minutes
Average	14 minutes

<i>Turnaround time using SJF (b)</i>	
<i>B</i>	4 minutes
<i>C</i>	8 minutes
<i>D</i>	12 minutes
<i>A</i>	20 minutes
Average	11 minutes

Figure 2-11 Turnaround time by running (a) the four jobs in First come first served order
(b) them in shortest job first order

- **Shortest Remaining Time Next** – is a preemptive version of shortest job first. With this algorithm, the scheduler always chooses the process whose remaining run time is the shortest. Again here, the run time has to be known in advance. When a new job arrives, its total time is compared to the current process' remaining time. If the new job needs less time to finish than the current process, the current process is suspended and the new job started. This scheme allows new short jobs to get good service.

Scheduling in Interactive Systems

- **Round-Robin Scheduling** –each process is assigned a time interval, called its **quantum**, which it is allowed to run. If the process is still running at the end of the quantum, the CPU is preempted and given to another process. If the process has blocked or finished before the quantum has elapsed, the CPU is switched to another process.

When using this algorithm, the length of the quantum should be set carefully. Because setting the quantum too short causes too many process switches and lowers the CPU efficiency and also setting it too long may cause poor response to short interactive requests. A quantum around 20-50 milliseconds is often a reasonable compromise.

Strength

- Simple to implement
- Fair

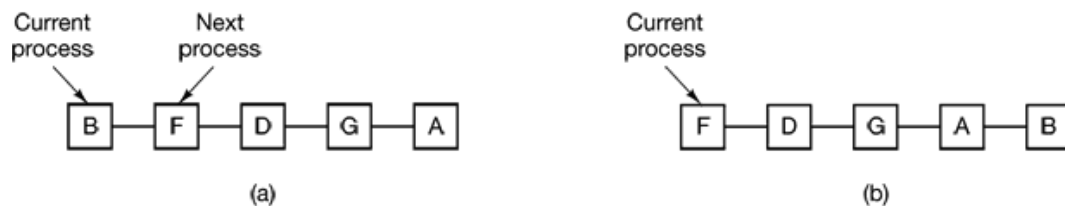


Figure 2-12 Round-robin scheduling. (a) The list of runnable processes

(b) The list of runnable processes after *B* uses up its

quantum.

- **Priority Scheduling** - each process is assigned a priority, and the runnable process with the highest priority is allowed to run. To prevent high-priority processes from running indefinitely, the scheduler may decrease the priority of the currently running process at each clock tick (i.e., at each clock interrupt). If this action causes its priority to drop below that of the next highest process, a process switch occurs.

Alternatively, each process may be assigned a maximum time quantum that it is allowed to run. When this quantum is used up, the next highest priority process is given a chance to run.

It is often convenient to group processes into priority classes and use priority scheduling among the classes but round-robin scheduling within each class. Figure 2-13 shows a system with four priority classes. The scheduling algorithm is as follows:

As long as there are runnable processes in priority class 4, just run each one for one quantum, round-robin fashion, and never bother with lower priority classes, if priority class 4 is empty, then run the class 3 processes round robin. If classes 4 and 3 are both empty, then run class 2 round robin, and so on. If priorities are not adjusted occasionally, lower priority classes may all starve to death.

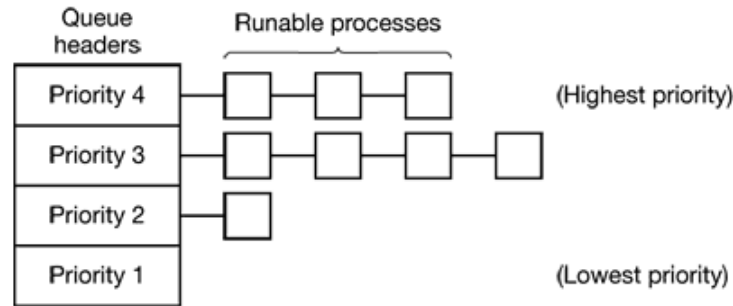


Figure 2-13 A scheduling algorithm with four priority classes

- **Multiple Queues** – several priority classes are set up and a mechanism is enforced such that processes in the highest class were run for one quantum, processes in the next highest class were run for two quanta, processes in the next class were run for four quanta, and so on. Whenever a process used up all the quanta allocated to it, it is moved down one class.

Strength

- Reduces CPU switch

- **Shortest Process Next** – the shortest process will be run first. The only problem is figuring out which of the currently runnable processes is the shortest one. A common approach to deal with this is to approximate its value based on past behavior. The next CPU burst is generally predicted as *an exponential average of the measured lengths of previous CPU bursts*. Let t_n be the length of the n^{th} CPU (which is the most recent CPU burst) and let T_{n+1} be our predicted value for the next CPU burst. Then, for $\alpha \leq 0 \leq 1$, an exponential average can be defined as

$$T_{n+1} = \alpha t_n + (1 - \alpha) T_n ; \quad \text{where } t_n \text{ contains our most recent information} \\ \text{and } T_n \text{ stores the past history.}$$

The parameter α controls the relative weight of recent and past history in our prediction.

- If $\alpha = 0$, then $T_{n+1} = T_n$ and recent history has no effect
- If $\alpha = 1$, then $T_{n+1} = t_n$ and only the most recent CPU burst matters

But more commonly, $\alpha = 1/2$, so recent history and past history are equally weighted.

Expanding the formula for T_{n+1} , we find :

$$T_{n+1} = \alpha t_n + (1 - \alpha) \alpha t_{n-1} + \dots + (1 - \alpha)^j \alpha t_{n-j} + (1 - \alpha)^{n+1} T_0$$

Since both α and $(1 - \alpha)$ are less than or equal to 1, each successive term has less weight than its predecessor.

- **Lottery Scheduling** - processes are given lottery tickets for various system resources, such as CPU time and whenever a scheduling decision has to be made, a lottery ticket is chosen at random, and the process holding that ticket gets the resource. When applied to CPU scheduling, the system might hold a lottery 50 times a second, with each winner getting 20 msec of CPU time as a prize. More important processes can be given extra tickets, to increase their odds of winning.

Strength

- Lottery scheduling is highly responsive (if a new process shows up and is granted some tickets, at the very next lottery it will have a chance of winning in proportion to the number of tickets it holds.)
- Cooperating processes may exchange tickets if they wish. For example, when a client process sends a message to a server process and then blocks, it may give all of its tickets to the server, to increase the chance of the server running next. When the server is finished, it returns the tickets so the client can run again.
-
- **Fair-Share Scheduling** - takes into account that owns a process before scheduling it. In this model, a user is allocated some fraction of the CPU and the scheduler picks processes in such a way as to enforce it. Thus if two users have each been promised 50% of the CPU, they will each get that, no matter how many processes they have in existence.

As an example, consider a system with two users, each of which has been promised 50% of the CPU. User 1 has four processes, *A*, *B*, *C*, and *D*, and user 2 has only 1 process, *E*. If round-robin scheduling is used, a possible scheduling sequence that meets all the constraints is :

A E B E C E D E A E B E C E D E ...

On the other hand, if user 1 is entitled to twice as much CPU time as user 2, we might get

A B E C D E A B E C D E ...

But if the normal round robin were to be used, user 1 would get 80% of the CPU time because the scheduling sequence would be

A B C D E A B C D E A B C D E ...

Scheduling in Real-Time Systems

A *real-time* system is one in which time plays an essential role. Typically, one or more physical devices external to the computer generate stimuli, and the computer must react appropriately to them within a fixed amount of time. Examples include a patient monitoring in a hospital intensive-care unit, the autopilot in an aircraft, and robot control in an automated factory. In all these cases, having the right answer but having it too late is often just as bad as not having it at all.

Real-time systems are generally categorized as:

- **Hard real time** – systems where absolute deadlines that must be met
- **Soft real time**- systems where missing an occasional deadline is undesirable, but nevertheless tolerable.

Real-time scheduling algorithms can be *static* or *dynamic*. Static algorithms assign each process a fixed priority in advance and then do prioritized preemptive scheduling using those priorities. Static scheduling only works when there is perfect information available in advance about the work needed to be done and the deadlines that have to be met. Dynamic scheduling algorithms do not have these restrictions.

- **Rate Monotonic Scheduling** – is a classic static real-time scheduling algorithm for preemptable, periodic processes. It works by assigning each process a fixed priority equal to the frequency of occurrence of its triggering event. For example, a process that must run every 30 msec (33 times/sec) gets priority 33, a process that must run every 40 msec (25 times/sec) gets priority 25,

and a process that must run every 50 msec (20 times/sec) gets priority 20.³ At run time, the scheduler always runs the highest priority ready process, preempting the running process if need be.

Figure 2.14 shows how rate monotonic scheduling works with three processes *A*, *B*, and *C* with static priorities, 33, 25, and 20, respectively (which means that whenever *A* needs to run, it runs, preempting any other process currently using the CPU. Process *B* can preempt *C*, but not *A*. Process *C* has to wait until the CPU is otherwise idle in order to run.)

If initially all three processes are ready to run. The highest priority one, *A*, is chosen, and allowed to run until it completes at 15 msec, as shown in the RMS line. After it finishes, *B* and *C* are run in that order. Together, these processes take 30 msec to run, so when *C* finishes, it is time for *A* to run again. This rotation goes on until the system goes idle at $t = 70$.

At $t = 80$ *B* becomes ready and runs. However, at $t = 90$, a higher priority process, *A*, becomes ready, so it preempts *B* and runs until it is finished, at $t = 100$. At that point the system can choose between finishing *B* or starting *C* so it chooses the highest priority process, *B*.

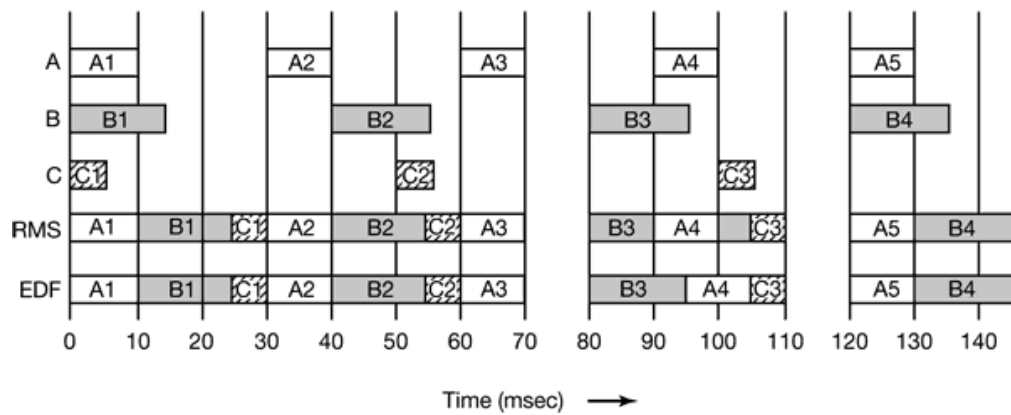


Figure 2.14 An example of RMS and EDF real-time scheduling

DEADLOCKS

Computer systems are full of resources that can only be used by one process at a time. Common examples include printers, tape drives, and slots in the system's internal tables. Having two processes simultaneously writing to the printer leads to gibberish. Having two processes using the same file system table slot will invariably lead to a corrupted file system. Consequently, all operating systems have the ability to (temporarily) grant a process exclusive access to certain resources.

³ The priorities are thus linear with the rate (number of times/second the process runs). This is why it is called rate monotonic.

For many applications, a process needs exclusive access to not one resource, but several. Suppose, for example, two processes (A and B) each want to record a scanned document on a CD. The following can happen.

- Process *A* requests permission to use the scanner and is granted it.
- Process *B* is programmed differently and requests the CD recorder first and is also granted it.
- Now *A* asks for the CD recorder, but the request is denied until *B* releases it.
- Instead of releasing the CD recorder, *B* asks for the scanner.

At this point both processes are blocked and will remain so forever. This situation is called a **deadlock**. Deadlock can be defined formally as follows:

A set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause. Because all the processes are waiting, none of them will ever cause any of the events that could wake up any of the other members of the set, and all the processes continue to wait forever.

Deadlocks can also occur across machines. For example, many offices have a local area network with many computers connected to it. Often devices such as scanners, CD recorders, printers, and tape drives are connected to the network as shared resources, available to any user on any machine. If these devices can be reserved remotely (i.e., from the user's home machine), the same kind of deadlocks can occur as described above. More complicated situations can cause deadlocks involving three, four, or more devices and users.

Deadlocks can occur in a variety of situations besides requesting dedicated I/O devices. In a database system, for example, a program may have to lock several records it is using, to avoid race conditions. If process *A* locks record *R1* and process *B* locks record *R2*, and then each process tries to lock the other one's record, we also have a deadlock. Thus deadlocks can occur on hardware resources or on software resources.

Resources

Deadlocks can occur when processes have been granted exclusive access to devices, files, and so forth. For general discussion, we will refer to the objects granted as **resources**. A resource can be a hardware device (e.g., a tape drive) or a piece of information (e.g., a locked record in a database).

Resources come in two types: preemptable and nonpreemptable.

- A *preemptable* resource is one that can be taken away from the process owning it with no ill effects. Memory is an example of a preemptable resource.
- A *nonpreemptable resource*, in contrast, is one that cannot be taken away from its current owner without causing the computation to fail. If a process has begun to burn a CD-ROM, suddenly taking the CD recorder away from it and giving it to another process will result in a garbled CD, CD recorders are not preemptable at an arbitrary moment.

Conditions for Deadlock

All of the following four conditions must hold for there to be a deadlock:

- Mutual exclusion condition. Each resource is either currently assigned to exactly one process or is available.
- Hold and wait condition. Processes currently holding resources granted earlier can request new resources.
- No preemption condition. Resources previously granted cannot be forcibly taken away from a process. They must be explicitly released by the process holding them.
- Circular wait condition. There must be a circular chain of two or more processes, each of which is waiting for a resource held by the next member of the chain.

If one of them is absent, no deadlock is possible.

Starvation

A problem closely related to deadlock is **starvation**. In a dynamic system, requests for resources happen all the time. Some policy is needed to make a decision about who gets which resource when. This policy, although seemingly reasonable, may lead to some processes never getting service even though they are not deadlocked.

As an example, consider allocation of the printer. Imagine that the system uses some kind of algorithm to ensure that allocating the printer does not lead to deadlock. Now suppose that several processes all want it at once. Which one should get it?

One possible allocation algorithm is to give it to the process with the smallest file to print (assuming this information is available). This approach maximizes the number of happy customers and seems fair. Now consider what happens in a busy system when one process has a huge file to print. Every time the printer is free, the system will look around and choose the process with the shortest file. If there is a constant stream of processes with short files, the process with the huge file will never be allocated the printer. It will simply *starve to death* (be postponed indefinitely, even though it is not blocked).

Starvation can be avoided by using a first-come, first-serve resource allocation policy. With this approach, the process waiting the longest gets served next. In due course of time, any given process will eventually become the oldest and thus get the needed resource.

Deadlock Modeling

Deadlock can be modeled using directed graphs. The graphs have two kinds of nodes: processes, shown as circles, and resources, shown as squares. An arc from a resource node (square) to a process node (circle) means that the resource has previously been requested by, granted to, and is currently held by that process. In Fig. 2-15 (a), resource *R* is currently assigned to process *A*.

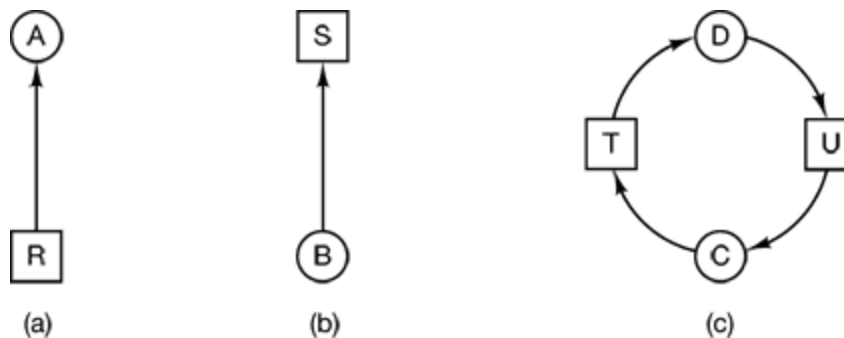


Figure 2-15 Resource allocation graphs. (a) Holding a resource. (b) Requesting a resource. (c) Deadlock.

An arc from a process to a resource means that the process is currently blocked waiting for that resource. In 2-15 (b), process *B* is waiting for resource *S*.

In Fig. 2-15 (c) we see a deadlock: process *C* is waiting for resource *T*, which is currently held by process *D*. Process *D* is not about to release resource *T* because it is waiting for resource *U*, held by *C*. Both processes will wait forever. A cycle in the graph means that there is a deadlock involving the processes and resources in the cycle (assuming that there is one resource of each kind). In this example, the cycle is *C–T–D–U–C*.

Using Resource Graphs

Resource graphs are a tool that let us see if a given request/release sequence leads to deadlock. We just carry out the requests and releases step by step, and after every step check the graph to see if it contains any cycles. If so, we have a deadlock; if not, there is no deadlock.

Imagine that we have three processes, *A*, *B*, and *C*, and three resources *R*, *S*, and *T*. The requests and releases of the three processes are given in Fig. 2-16 (a)-(c). The operating system is free to run any unblocked process at any instant, so it could decide to run *A* until *A* finished all its work, then run *B* to completion, and finally run *C*. This ordering does not lead to any deadlocks (because there is no competition for resources) but it also has no parallelism at all.

Let us now suppose that the processes do both I/O and computing, so that round robin is a reasonable scheduling algorithm. The resource requests might occur in the order of Fig. 2-16 (d). If these six requests are carried out in that order, the six resulting resource graphs are shown in Fig. 2-16 (e)-(j). After request 4 has been made, *A* blocks waiting for *S*, as shown in Fig. 2-16 (h). In the next two steps, *B* and *C* also block, ultimately leading to a cycle and the deadlock of Fig. 2-16 (j).

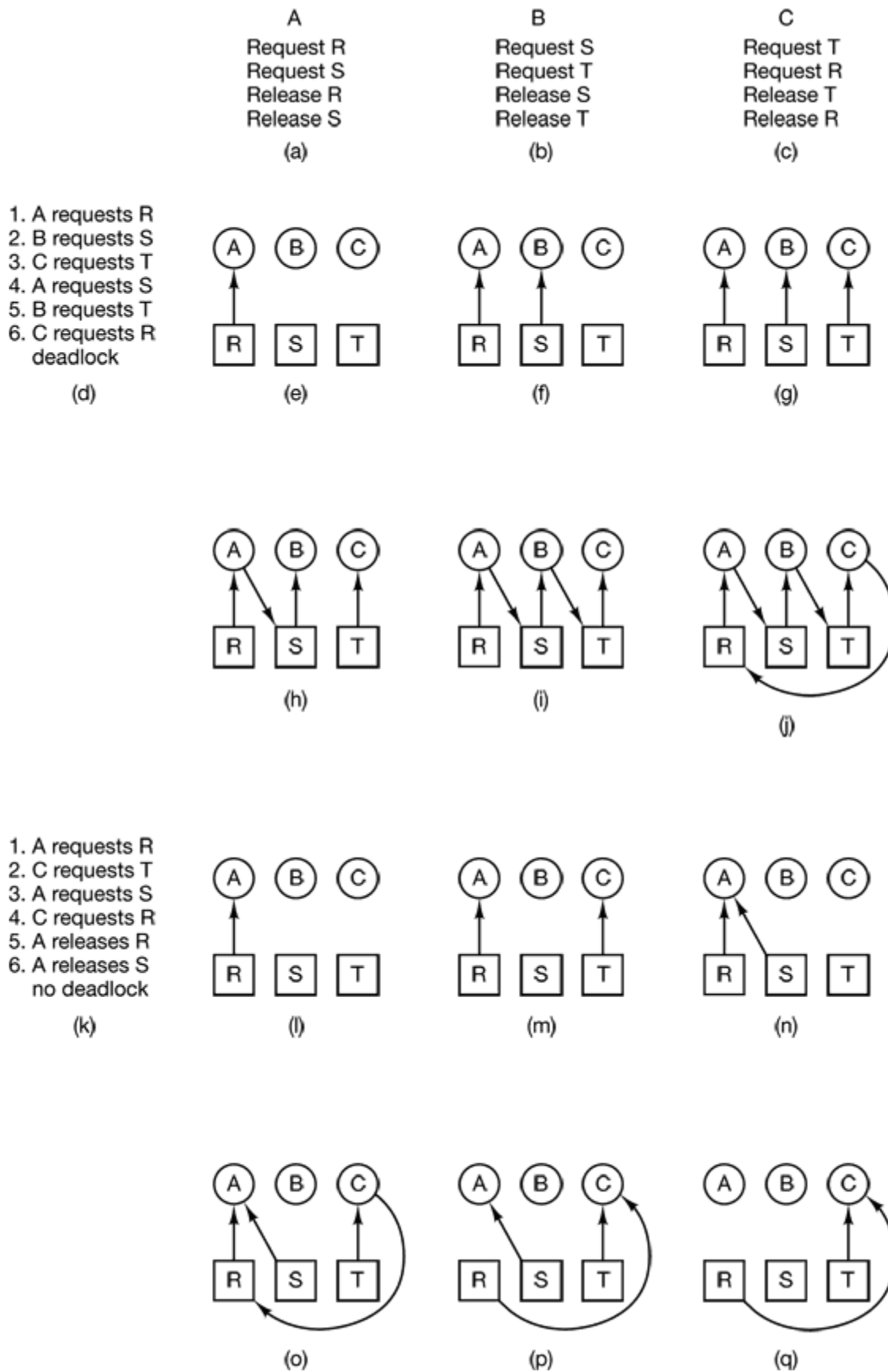


Figure 2-16 An example of how deadlock occurs and how it can be avoided

However, as we have already mentioned, the operating system is not required to run the processes in any special order. In particular, if granting a particular request might lead to deadlock, the operating system can simply suspend the process without granting the request (i.e., just not schedule the process) until it is safe. In Fig. 2-16, if the operating system knew about the impending deadlock, it could suspend *B* instead of granting it *S*. By running only *A* and *C*, we would get the requests and releases of Fig. 2-16 (k) instead of Fig. 2-16 (d). This sequence leads to the resource graphs of Fig. 2-16 (l)-(q), which do not lead to deadlock.

After step (q), process *B* can be granted *S* because *A* is finished and *C* has everything it needs. Even if *B* should eventually block when requesting *T* no deadlock can occur. *B* will just wait until *C* is finished. Although our treatment of resource graphs has been for the case of a single resource of each type, resource graphs can also be generalized to handle multiple resources of the same type.

Dealing with Deadlocks

In general, four strategies are used for dealing with deadlocks. These are:

- Just ignoring the problem altogether (the Ostrich Algorithm)
- Detection and recovery (let deadlocks occur, detect them, and take action.)
- Dynamic avoidance by careful resource allocation.
- Prevention, by structurally negating one of the four conditions necessary to cause a deadlock.

We will examine each of these methods in turn in the next sections.

The Ostrich Algorithm –is the simplest approach in which you stick your head in the sand and pretend there is no problem at all.

Suppose that a UNIX system has 100 process slots. Ten programs are running, each of which needs to create 12 child processes. After each process has created 9 processes, the 10 original processes and the 90 new processes have exhausted the table. Each of the 10 original processes now sits in an endless loop forking and failing—a deadlock. In this algorithm, if a fork fails because the table is full, a reasonable approach for the program doing the fork is to wait a random time and try again.

Deadlock Detection and Recovery- when this technique is used, the system does not attempt to prevent deadlocks from occurring. Instead, it lets them occur, tries to detect when this happens, and then takes some action to recover after the fact. One of the ways to detect deadlocks is the use of resource graph.

- **Resource graph method of deadlock detection** – construct the graph and see if this graph contains a cycle. If there is one or more cycles, then a deadlock exists. Any process that is part of a cycle is deadlocked. If no cycles exist, the system is not deadlocked. For example, consider the following scenario of seven processes, *A* through *G*, and six resources, *R* through *W*. The state of which resources are currently owned and which ones are currently being requested is as follows:

Process *A* holds *R* and wants *S*.
 Process *B* holds nothing but wants *T*.
 Process *C* holds nothing but wants *S*.
 Process *D* holds *U* and wants *S* and *T*.
 Process *E* holds *T* and wants *V*.
 Process *F* holds *W* and wants *S*.
 Process *G* holds *V* and wants *U*.

The question is: “Is this system deadlocked, and if so, which processes are involved?” To answer this question we will construct a resource graph as shown in Figure 2-17. This graph contains one cycle (shown in Fig. 3-5(b)), which can be seen by visual inspection. From this cycle, we can see that processes *D*, *E*, and *G* are all deadlocked. Processes *A*, *C*, and *F* are not deadlocked because *S* can be allocated to any one of them, which then finishes and returns it. Then the other two can take it in turn and also complete.

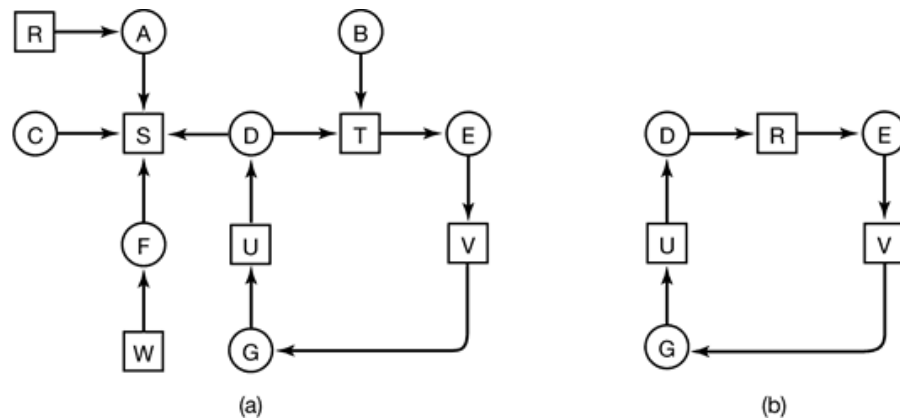


Figure 2-17 (a) A resource graph. (b) A cycle extracted from (a)

Once a deadlock is detected in our system, some way is needed to recover and get the system going again. Some of the techniques to recover from deadlock are:

- Recovery through Preemption** - temporarily take a resource away from its current owner and give it to another process. The ability to take a resource away from a process, have another process use it, and then give it back without the process noticing it is highly dependent on the nature of the resource. Recovering this way is frequently difficult or impossible. Choosing the process to suspend depends largely on which ones have resources that can easily be taken back.
- Recovery through Rollback**- arrange to have processes *checkpointed* periodically and recover the system from deadlock by rolling back the process that owns a needed resource to a point in time before it acquired the resource (starting one of its earlier checkpoints)
- Recovery through Killing Processes** - carefully chose and kill a process in or not the cycle in the deadlock cycle so that the resources will be released and the deadlock broken. Where possible, it is best to kill a process that can be rerun from the beginning with no ill effects. For example, a compilation can always be rerun

because all it does is read a source file and produce an object file. If it is killed part way through, the first run has no influence on the second run. On the other hand, a process that updates a database cannot always be run a second time safely. If the process adds 1 to some record in the database, running it once, killing it, and then running it again will add 2 to the record, which is incorrect.

Deadlock Avoidance - decide whether granting a resource is safe or not and only make the allocation when it is safe. A state is said to be safe if it is not deadlocked and there is some scheduling order in which every process can run to completion even if all of them suddenly request their maximum number of resources immediately.

Consider a situation where we have one resource and three processes - *A*, *B*, and *C* (Fig. 2-18 (a)). We have a state in which *A* has 3 instances of the resource but may need as many as 9 eventually. *B* currently has 2 and may need 4 altogether, later. Similarly, *C* also has 2 but may need an additional 5.

Has Max		
A	3	9
B	2	4
C	2	7
Free: 3		
(a)		

Has Max		
A	3	9
B	4	4
C	2	7
Free: 1		
(b)		

Has Max		
A	3	9
B	0	—
C	2	7
Free: 5		
(c)		

Has Max		
A	3	9
B	0	—
C	7	7
Free: 0		
(d)		

Has Max		
A	3	9
B	0	—
C	0	—
Free: 7		
(e)		

Figure 2-18 Demonstrating that the state in (a) is safe

If a total of 10 instances of the resource exist, with 7 resources already allocated, there are 3 still free. This indicates that the state of Fig. 2-18 (a) is safe because there exists a sequence of allocations that allows all processes to complete. Namely, the scheduler could simply run *B* exclusively, until it asked for and got two more instances of the resource, leading to the state of Fig. 2-18 (b). When *B* completes, we get the state of Fig. 2-18 (c). Then the scheduler can run *C* leading eventually to Fig. 2-18 (d). When *C* completes, we get Fig. 2-18 (e). Now *A* can get the six instances of the resource it needs and also complete. Thus the state of Fig. 2-18 (a) is safe because the system, by careful scheduling, can avoid deadlock.

Now suppose we have the initial state shown in Fig. 2-19 (a), but this time *A* requests and gets another resource, giving Fig. 2-19 (b). Then the scheduler could run *B* until it asked for all its resources, as shown in Fig. 2-19 (c). Eventually, *B* completes and we get the situation of Fig. 2-19 (d). At this point we are stuck. We only have four instances of the resource free, and each of the active processes needs five. There is no sequence that guarantees completion. Thus the allocation decision that moved the system from Fig. 2-19 (a) to Fig. 2-19 (b) went from a safe state to an unsafe state.

Running *A* or *C* next starting at Fig. 2-19 (b) does not work either. In retrospect, *A*'s request should not have been granted.

Has Max		
A	3	9
B	2	4
C	2	7
Free: 3		
(a)		

Has Max		
A	4	9
B	2	4
C	2	7
Free: 2		
(b)		

Has Max		
A	4	9
B	4	4
C	2	7
Free: 0		
(c)		

Has Max		
A	4	9
B	—	—
C	2	7
Free: 4		
(d)		

Figure 2-19 Demonstrating that the state in (b) is not safe

A scheduling algorithm that can avoid deadlocks is due to Dijkstra (1965) and is known as the **banker's algorithm**. It is modeled on the way a small-town banker might deal with a group of customers to whom he has granted lines of credit. What the algorithm does is check to see if granting the request leads to an unsafe state. If it does, the request is denied. If granting the request leads to a safe state, it is carried out. In Fig. 2-20 (a) we see four customers. *A*, *B*, *C* and *D*, each of whom has been granted a certain number of credit units (e.g., 1 unit is 1K dollars). The banker knows that not all customers will need their maximum credit immediately, so he has reserved only 10 units rather than 22 to service them. (In this analogy, customers are processes, units are, say, tape drives, and the banker is the operating system.)

Has Max		
A	0	6
B	0	5
C	0	4
D	0	7
Free: 10		
(a)		

Has Max		
A	1	6
B	1	5
C	2	4
D	4	7
Free: 2		
(b)		

Has Max		
A	1	6
B	2	5
C	2	4
D	4	7
Free: 1		
(c)		

Figure 2-20 Three resource allocation states: (a) Safe. (b) Safe (c) Unsafe.

The customers go about their respective businesses, making loan requests from time to time (i.e., asking for resources). At a certain moment, the situation is as shown in Fig. 2-20 (b). This state is safe because with two units left, the banker can delay any requests except *C*'s, thus letting *C* finish and release all four of his resources. With four units in hand, the banker can let either *D* or *B* have the necessary units, and so on. However if a request from *B* for one more unit were granted in Fig. 2-20 (b). We would have situation unsafe in Fig. 2-20 (c).

But in general, deadlock avoidance is essentially impossible. Because it requires information about future requests, which is not known.

Deadlock Prevention – make deadlocks structurally impossible by ensuring that at least one of these conditions is never satisfied. This involves:

- *Attacking the Mutual Exclusion Condition*- never assigned exclusively to a single process. But this can lead to chaos.
- *Attacking the Hold and Wait Condition* - prevent processes that hold resources from waiting for more resources. This can be done by forcing a process requesting a resource to first temporarily release all the resources it currently holds.
- *Attacking the No Preemption Condition* – take a resource from the process forcibly and give to another process. But this is not always a good choice.
- *Attacking the Circular Wait Condition* - provide a global numbering of all the resources, as shown in Fig. 2-21(a) and make sure that processes can request resources whenever they want to, but all requests must be made in numerical order(A process may request first a printer and then a tape drive, but it may not request first a plotter and then a printer.)

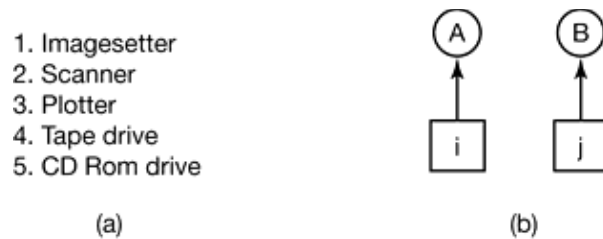


Figure 2-21 (a) Numerically ordered resources. (b) A resource graph.

With this technique, the resource allocation graph can never have cycles. In Figure 2-21(b). We can get a deadlock only if A requests resource j and B requests resource i . Assuming i and j are distinct resources, they will have different numbers. If $i > j$, then A is not allowed to request j because that is lower than what it already has. If $i < j$, then B is not allowed to request i because that is lower than what it already has. Either way, deadlock is impossible.